



Class Notes: Object-Oriented Programming Concepts (Sessions 1 & 2) - Java

Session 1: Introduction to Object-Oriented Programming

Duration: 2 hours

Objective: Understand the OOP paradigm, differentiate it from other programming approaches, and introduce foundational concepts like classes, objects, and methods.

Overview

- **What is OOP?:** Object-Oriented Programming (OOP) is a programming paradigm that organizes code around objects, which are instances of classes combining data (fields) and behavior (methods). It promotes modularity, reusability, and maintainability.
- **OOP vs. Object-Based Programming:**
 - **OOP:** Supports encapsulation, abstraction, inheritance, and polymorphism (e.g., Java).
 - **Object-Based:** Supports objects and encapsulation but lacks inheritance and polymorphism (e.g., older versions of JavaScript).
- **OOP vs. Procedural Programming:**
 - **Procedural:** Focuses on procedures/functions and sequential execution (e.g., C).
 - **OOP:** Focuses on objects modeling real-world entities and their interactions.
- **Object-Oriented Design (OO Design):** Designing software by modeling real-world entities as classes with fields and methods, emphasizing scalability and modularity.
- **Responsibility-Driven Design (RDD):** Assigning specific responsibilities to classes to ensure clear roles and interactions.
- **Key Concepts:**
 - **Class:** A blueprint for objects (e.g., a `Car` class defines fields like `color` and methods like `drive`).
 - **Object:** An instance of a class (e.g., a specific car with `color = "red"`).

- **Method:** A function defined within a class to describe object behavior.
- **Agent:** An object that performs actions in a system.
- **Module:** A cohesive unit of code, such as a class or package in Java.

Activity 1: Group Discussion (15 minutes)

- **Task:** In groups of 3–4, discuss a real-world system (e.g., a library, bank, or school) and identify potential objects, their fields, and methods.
- **Example:** For a library system:
 - Object: Book
 - Fields: `title` , `author` , `isbn`
 - Methods: `borrow()` , `returnBook()`
- **Outcome:** Each group presents one object and its properties, fostering understanding of OOP modeling in Java.

Use Case: Library Management System

- **Scenario:** A library needs a system to manage books and members.
- **OOP Approach in Java:**
 - Classes: `Book` , `Member` , `Librarian`
 - Objects: A specific book (e.g., "Java Programming" with ISBN "12345"), a specific member (e.g., John Doe)
 - Methods: `Book.borrow()` , `Member.register()`
- **Benefit:** OOP allows intuitive modeling of real-world entities, making the system scalable and maintainable.

Example 1: Simple Class in Java

```
public class Car {  
    // Fields  
    String brand;  
    int speed;  
  
    // Method  
    public void display() {  
        System.out.println("Brand: " + brand + ", Speed: " + speed + " km/h");  
    }  
  
    // Main method for testing  
    public static void main(String[] args) {  
        Car car1 = new Car(); // Object creation  
        car1.brand = "Toyota";  
        car1.speed = 120;  
        car1.display();  
    }  
}
```

- **Output:** Brand: Toyota, Speed: 120 km/h
- **Explanation:** The `Car` class defines a blueprint with fields (`brand` , `speed`) and a method (`display`). `car1` is an object instantiated from the class.

Classwork 1: Create a Class (20 minutes)

- **Task:** In pairs, write a Java program to define a `Student` class with fields (`name` , `rollNumber`) and a method to display student details.
- **Instructions:**
 - i. Create the `Student` class with public fields and a display method.
 - ii. Instantiate two `Student` objects and call the display method.
 - iii. Share your code with the class.
- **Sample Solution:**

```

public class Student {
    public String name;
    public int rollNumber;

    public void display() {
        System.out.println("Name: " + name + ", Roll Number: " + rollNumber);
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Alice";
        s1.rollNumber = 101;
        s1.display();

        Student s2 = new Student();
        s2.name = "Bob";
        s2.rollNumber = 102;
        s2.display();
    }
}

```

- **Output:**

```

Name: Alice, Roll Number: 101
Name: Bob, Roll Number: 102

```

Session 2: Classes, Objects, and Encapsulation

Duration: 2 hours

Objective: Learn to create classes and objects in Java and implement encapsulation to protect data.

Overview

- **Classes and Objects:**
 - **Class:** Defines the structure (fields) and behavior (methods) of objects.
 - **Object:** A specific instance of a class with actual values.

- **Static Fields:** Shared across all objects of a class (e.g., a counter for the number of objects).
- **Final Fields:** Immutable values defined at compile time (equivalent to constant fields).
- **Encapsulation:**
 - Bundling data and methods within a class and controlling access using access modifiers (`public` , `private` , `protected`).
 - **Getters:** Methods to retrieve private fields.
 - **Setters:** Methods to modify private fields safely.
- **Benefits:**
 - Data hiding prevents unauthorized access.
 - Ensures data integrity through controlled modification.

Activity 2: Real-World Object Modeling (15 minutes)

- **Task:** Individually, identify an object from a hospital system (e.g., `Patient` , `Doctor`) and list its fields and methods. Discuss how encapsulation could protect sensitive data (e.g., patient records).
- **Example:**
 - Object: `Patient`
 - Fields: `name` , `medicalID` (private), `diagnosis` (private)
 - Methods: `getDiagnosis()` , `setDiagnosis()`
- **Outcome:** Reinforces modeling real-world entities and the importance of data protection in Java.

Use Case: Bank Account Management

- **Scenario:** A bank needs to manage customer accounts securely.
- **OOP Approach in Java:**
 - Class: `BankAccount`
 - Fields: `accountNumber` (private), `balance` (private)
 - Methods: `deposit()` , `withdraw()` , `getBalance()`
- **Encapsulation:** Private fields ensure `balance` cannot be modified directly, only through methods with validation.

Example 2: Encapsulation in Java

```
public class BankAccount {
    private String accountNumber;
    private double balance;

    // Constructor
    public BankAccount(String accountNumber, double initialBalance) {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
    }

    // Getter
    public double getBalance() {
        return balance;
    }

    // Setter
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited $" + amount + ". New balance: $" + balance);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrew $" + amount + ". New balance: $" + balance);
        } else {
            System.out.println("Invalid withdrawal amount or insufficient funds.");
        }
    }

    public static void main(String[] args) {
        BankAccount account = new BankAccount("12345", 1000.0);
        account.deposit(500.0);
        account.withdraw(200.0);
        System.out.println("Final balance: $" + account.getBalance());
    }
}
```

```
}  
}
```

- **Output:**

```
Deposited $500.0. New balance: $1500.0  
Withdrew $200.0. New balance: $1300.0  
Final balance: $1300.0
```

- **Explanation:** The `BankAccount` class uses `private` fields to encapsulate data, with public methods to control access and ensure valid operations.

Classwork 2: Implement Encapsulation (25 minutes)

- **Task:** In groups of 2–3, write a Java program to define a `Rectangle` class with encapsulated fields (`length` , `width`) and methods to calculate area and perimeter.
- **Instructions:**
 - i. Use `private` for fields and `public` for methods.
 - ii. Include getters and setters with validation (e.g., `length` and `width` must be positive).
 - iii. Test the class by creating two `Rectangle` objects.
- **Sample Solution:**


```
public class Rectangle {
    private double length;
    private double width;

    public Rectangle(double length, double width) {
        setLength(length);
        setWidth(width);
    }

    public void setLength(double length) {
        if (length > 0) {
            this.length = length;
        } else {
            this.length = 1.0; // Default value
        }
    }

    public void setWidth(double width) {
        if (width > 0) {
            this.width = width;
        } else {
            this.width = 1.0; // Default value
        }
    }

    public double getLength() {
        return length;
    }

    public double getWidth() {
        return width;
    }

    public double getArea() {
        return length * width;
    }

    public double getPerimeter() {
        return 2 * (length + width);
    }
}
```

```

    }

    public static void main(String[] args) {
        Rectangle rect1 = new Rectangle(5.0, 3.0);
        Rectangle rect2 = new Rectangle(4.0, 6.0);

        System.out.println("Rectangle 1 - Area: " + rect1.getArea() + ", Perimeter: " + rect1.getPerimeter());
        System.out.println("Rectangle 2 - Area: " + rect2.getArea() + ", Perimeter: " + rect2.getPerimeter());
    }
}

```

- **Output:**

```

Rectangle 1 - Area: 15.0, Perimeter: 16.0
Rectangle 2 - Area: 24.0, Perimeter: 20.0

```

Scenario: Online Shopping System

- **Context:** An e-commerce platform needs to manage products and customers.
- **OOP Design in Java:**
 - **Class:** Product
 - Fields: `productID` (private), `name`, `price` (private)
 - Methods: `getPrice()`, `setPrice()`, `displayProduct()`
 - **Class:** Customer
 - Fields: `customerID` (private), `name`, `email` (private)
 - Methods: `getEmail()`, `setEmail()`, `placeOrder()`
- **Encapsulation:** Sensitive data like `productID` and `email` are private, accessible only through controlled methods.
- **Activity:** Discuss in class how encapsulation protects customer data (e.g., `email`) and ensures valid price updates.

Class Test: Sessions 1 & 2 (30 minutes)

- **Instructions:** Answer all questions. Write code in Java. Submit answers on paper or via Onlinevarsity.
- **Questions:**
 - Conceptual (5 points):**

- Define OOP and list its four main pillars. (2 points)
- Explain the difference between OOP and procedural programming with an example. (3 points)

ii. **Short Answer (5 points):**

- What is encapsulation, and why is it important? Provide a real-world analogy (e.g., a locked safe). (3 points)
- Differentiate between a class and an object with an example in Java. (2 points)

iii. **Coding (10 points):**

- Write a Java program to define a `Laptop` class with private fields `model` (String) and `price` (double). Include a constructor, getters, setters (with validation for positive price), and a method to display laptop details. Create two objects and demonstrate their use. (10 points)

• **Sample Solution for Question 3 (Java):**

```
public class Laptop {
    private String model;
    private double price;

    public Laptop(String model, double price) {
        this.model = model;
        setPrice(price);
    }

    public void setPrice(double price) {
        if (price > 0) {
            this.price = price;
        } else {
            this.price = 100.0; // Default price
        }
    }

    public double getPrice() {
        return price;
    }

    public String getModel() {
        return model;
    }

    public void display() {
        System.out.println("Model: " + model + ", Price: $" + price);
    }

    public static void main(String[] args) {
        Laptop laptop1 = new Laptop("Dell XPS", 1200.0);
        Laptop laptop2 = new Laptop("HP Spectre", 1500.0);

        laptop1.display();
        laptop2.display();
    }
}
```

- **Output:**

Model: Dell XPS, Price: \$1200.0

Model: HP Spectre, Price: \$1500.0

- **Grading Criteria:**
 - Conceptual: Clarity and correctness (5 points).
 - Short Answer: Accuracy and relevance (5 points).
 - Coding: Correct syntax, encapsulation, and functionality (10 points).

Additional Notes

- **Resources** (Page 5):
 - Use Onlinevarsity for eBook access, glossary, FAQs, and practice tests.
 - Submit lab assignments (e.g., Classwork 1 and 2) via Atrack for faculty evaluation.
- **References** (Page 11):
 - *Sams Teach Yourself Object Oriented Programming in 21 Days* by Anthony Sintes.
 - *An Introduction To Object Oriented Programming, 3/e* by Timothy Budd.
- **Feedback:** Submit feedback on course material via Onlinevarsity's feedback feature.
- **Next Steps:** Review classwork and test submissions. Prepare for Session 3 on abstraction by reading relevant eBook sections.